

University of Hertfordshire hosted by Global
Academic Foundation
School of Engineering and Computer Science

BSc Hons Computer Science (Artificial Intelligence)

6FTC2089 - Artificial Intelligence Project

Final Report

May 2026

Smart Vision Glasses for the Visually Impaired

Hanin Osama Saad Hussein Aly

Supervised by: Dr Lamees Nasser

Abstract

This project presents the design and development of an AI-powered smart vision assistance system aimed at helping visually impaired individuals navigate and understand their surroundings more independently. The system integrates real-time object detection using YOLOv8n, optical character recognition using EasyOCR, and spoken audio feedback via a text-to-speech module into a single unified pipeline, deployed on a Raspberry Pi 5 with an ultrasonic sensor for obstacle awareness.

The software was developed and tested using six indoor and outdoor images covering a range of real-world environments. Object detection achieved a precision of 73.6%, a recall of 90.7%, and an F1-score of 81.3%, with an average processing time of 1.15 seconds per image, meeting the sub-two-second response target set at the start of the project. OCR successfully extracted text from both clear and noisy images, and the text-to-speech module generated audio output for all 53 detections.

Hardware integration was partially completed. The Raspberry Pi 5 was connected to the camera module and ultrasonic sensor, though live camera capture could not be fully resolved due to a compatibility issue with the libcamera stack on the Pi 5. These limitations are discussed and evaluated honestly within the report.

The project demonstrates that combining affordable AI tools and embedded hardware into a functional assistive prototype is achievable at student level and provides a solid foundation for future wearable deployment.

Abstract	2
Glossary	3
Chapter 1 – Introduction	5
1.1 Background	6
1.2 Problem Statement	6
1.3 Aim and Objectives	7
Aim	7
Objectives	7
1.4 Project Scope.....	7
1.5 Proposed Methodology	8
1.6 Report Structure	8
Chapter 2 – Literature Review	10
2.1 Introduction.....	10
2.2 Real-Time Object and Text Recognition Using YOLO and OCR	10
2.3 Comparing YOLO Model Variants for Embedded Deployment	10
2.4 User Acceptance of AI-Powered Smart Vision Glasses	11
2.5 Evaluating Smart Vision Glasses with Totally Blind Users	11
2.6 Summary	12
Chapter 3 – System Requirements and Design	13
3.1 Introduction.....	14
3.2 Functional Requirements	15
3.3 Non-Functional Requirements	15
3.4 Hardware Requirements	15
3.5 Software Requirements	16
3.6 System Design	17
3.7 Design Challenges and Constraints	18
3.8 Summary	19
Chapter 4 – Methodology and Implementation	19
4.1 Introduction.....	20
4.2 Development Methodology	20
4.3 Object Detection Implementation.....	20
4.4 OCR Implementation.....	21
4.5 Text-to-Speech Implementation	22
4.6 Image Processing and OpenCV Integration	22
4.7 Raspberry Pi and Hardware Integration	23

4.8 System Workflow.....	24
4.9 Implementation Challenges	24
4.10 Summary	24
Chapter 5 – Hardware Integration	25
5.1 Introduction.....	26
5.2 Raspberry Pi 5 Setup	26
5.3 Camera Module Integration	27
5.4 Ultrasonic Sensor Integration	27
5.5 Audio Output Integration	28
5.6 Hardware Assembly and Prototype Design	28
5.7 Hardware Integration Challenges	29
5.8 Hardware Limitations	30
5.9 Summary	30
Chapter 6 – Testing, Results, Discussion and Evaluation	31
6.1 Introduction.....	32
6.2 Testing Methodology	32
6.2.1 Target Detection Classes	33
6.3 Object Detection Results	33
6.3.1 Evaluation Metrics	33
6.4 OCR Testing Results	36
6.5 Text-to-Speech Evaluation	37
6.6 Hardware Testing and Deployment Behaviour	37
6.7 Discussion	38
6.8 System Evaluation	39
6.9 Limitations	39
6.10 Summary	39
Chapter 7 – Conclusions and Future Work	40
7.1 Conclusion.....	41
7.2 Future Work.....	41
Chapter 8 – Project Management Review	42
8.1 Introduction.....	43
8.2 Project Planning and Time Management.....	43
8.3 Challenges Encountered.....	43
8.4 Adaptations and Changes to the Original Plan.....	44
8.5 Reflection on Personal Performance	44

8.6 Chapter Summary	44
References	44
APPENDIX A: Gantt Chart.....	45
Appendix B: Project Resources Checklist	46
Appendix C: Ethics Checklist.....	47
Appendix D: Risk Register	48

Glossary

Term	Meaning
AI	Artificial Intelligence — the simulation of human intelligence processes by machines
CNN	Convolutional Neural Network — a type of deep learning model designed for image recognition tasks
COCO	Common Objects in Context — a large-scale dataset used to train object detection models including YOLOv8
CSI	Camera Serial Interface — the ribbon cable port used to connect the Pi Camera module to the Raspberry Pi
EasyOCR	Open-source Python library that uses deep learning for optical character recognition across varied fonts and lighting
F1-Score	Harmonic mean of Precision and Recall, used as a single balanced measure of detection performance
FN	False Negative — a real object present in the image that the system failed to detect
FP	False Positive — an object detected by the system that was not actually present or relevant
FPS	Frames Per Second — a measure of how many image frames are processed per second
GPIO	General Purpose Input/Output — programmable pins on the Raspberry Pi used to connect hardware components
gTTS	Google Text-to-Speech — an online Python library that converts text into spoken MP3 audio
HC-SR04	A low-cost ultrasonic distance sensor used in the prototype for obstacle proximity detection
IoT	Internet of Things — physical devices embedded with electronics that can collect and exchange data
libcamera	The default camera software stack used by the Raspberry Pi 5 for image capture
MicroSD	Removable storage card used to hold the Raspberry Pi operating system and project files
OCR	Optical Character Recognition — technology that detects and converts printed text from images into machine-readable text
OpenCV	Open Source Computer Vision Library — used for image loading, processing and frame manipulation in Python
Python	High-level programming language used to develop all software components in this project
Raspberry Pi	A low-cost single-board computer used as an embedded computing platform

Raspberry Pi 5	The specific model of Raspberry Pi used as the main processing unit in this prototype
TP	True Positive — an object correctly detected and identified by the system
TTS	Text-to-Speech — technology that converts written text into spoken audio output
Ultrasonic Sensor	A sensor that uses sound waves to measure the distance between the device and nearby obstacles
YOLO	You Only Look Once — a family of real-time object detection neural network architectures
YOLOv8	A recent version of the YOLO architecture developed by Ultralytics, used in this project
YOLOv8n	The nano variant of YOLOv8 — the smallest and fastest version, chosen for Raspberry Pi deployment

Chapter 1 – Introduction

1.1 Background

Visual impairment affects millions of people worldwide and can significantly reduce a person's ability to navigate environments independently and safely. Everyday tasks such as identifying nearby objects, reading signs, recognising obstacles, and understanding surroundings may become difficult without assistance. Traditional mobility aids, including white canes and guide dogs, remain important tools for visually impaired individuals; however, these solutions can still have limitations in providing detailed environmental awareness and real-time information.

Recent advancements in Artificial Intelligence (AI), computer vision, and embedded systems have created new opportunities for assistive technologies. Deep learning models are now capable of recognising objects, analysing scenes, and detecting text in real time with increasingly high accuracy. At the same time, low-cost embedded hardware such as the Raspberry Pi has made it possible to develop portable and affordable smart systems for everyday use.

This project focuses on the development of a smart vision assistance prototype that combines object detection, optical character recognition (OCR), and audio feedback to support visually impaired users. The system uses AI-based computer vision techniques to identify important objects within the user's surroundings and convert detected information into spoken audio output. The project also explores the integration of hardware components including a Raspberry Pi camera module and ultrasonic sensors to create a functional assistive prototype.

The project was initially proposed as a fully wearable smart glasses system. However, due to hardware integration challenges and time limitations, the final implementation was developed as a functional table-mounted prototype that demonstrates the complete system architecture and core assistive features.

1.2 Problem Statement

Visually impaired individuals often face challenges when navigating unfamiliar environments or identifying nearby objects and text without assistance. Although existing assistive technologies can provide support, many commercial smart vision systems remain expensive, difficult to access, or heavily dependent on specialised hardware.

In addition, some traditional assistive tools provide limited contextual information and may not offer real-time object recognition or text reading capabilities. There is therefore a need for an affordable and accessible assistive system capable of detecting surrounding objects, recognising text, and providing immediate audio feedback to users.

This project aims to address this problem by developing an AI-powered smart vision prototype using computer vision and embedded system technologies. The proposed system is intended to assist users by improving environmental awareness through automated object detection, OCR functionality, and speech-based feedback.

1.3 Aim and Objectives

Aim

The aim of this project is to develop a smart AI-based assistive vision system capable of detecting objects, recognising text, and providing audio feedback to support visually impaired users.

Objectives

The main objectives of the project are:

- To develop a real-time object detection system using the YOLOv8 deep learning model.
- To implement Optical Character Recognition (OCR) for reading printed text from images.
- To integrate text-to-speech functionality for spoken audio feedback.
- To deploy the system using Raspberry Pi hardware and connected peripherals.
- To evaluate the performance of the system using object detection metrics, OCR testing, and processing time analysis.
- To investigate the practical limitations and challenges associated with embedded AI deployment.

1.4 Project Scope

The scope of this project includes the design, implementation, and evaluation of an AI-powered assistive vision prototype. The system focuses primarily on detecting common indoor and outdoor objects, recognising visible printed text, and converting outputs into speech-based feedback.

The software implementation includes:

- YOLOv8 object detection
- OCR using EasyOCR
- Text-to-speech integration
- Image processing using OpenCV

The hardware implementation includes:

- Raspberry Pi 5
- Raspberry Pi Camera Module
- HC-SR04 ultrasonic sensor
- Breadboard and wiring connections

The project does not aim to create a fully commercial wearable product. Instead, the final outcome is a functional prototype demonstrating the integration of AI and embedded systems for assistive technology applications.

Several limitations affected the project scope, including hardware debugging challenges, Raspberry Pi camera integration issues, and limited time available for complete wearable assembly and large-scale real-world testing.

1.5 Proposed Methodology

The project followed a practical development methodology consisting of research, system design, implementation, testing, and evaluation stages. Existing research related to assistive technologies, object detection models, OCR systems, and embedded AI applications was first reviewed to identify suitable approaches and technologies.

The software system was then implemented using Python, YOLOv8, OpenCV, EasyOCR, and text-to-speech libraries. Object detection and OCR modules were integrated into a unified processing pipeline capable of analysing captured images and generating audio feedback.

Hardware integration was performed using a Raspberry Pi 5 connected to a camera module and ultrasonic sensor. Testing was conducted using multiple indoor and outdoor images to evaluate object detection performance, OCR functionality, processing time, and system behaviour under different conditions.

The final stage involved analysing the system's strengths, limitations, and potential future improvements for assistive smart vision technologies.

1.6 Report Structure

This report is organised into several chapters.

Chapter 1 introduces the background, problem statement, aims, objectives, and scope of the project.

Chapter 2 presents a literature review covering assistive technologies, computer vision systems, object detection models, OCR technologies, and embedded AI applications.

Chapter 3 discusses the system requirements, architecture, and overall design of the proposed prototype.

Chapter 4 explains the methodology and implementation process, including the development of the object detection, OCR, text-to-speech, and Raspberry Pi integration components.

Chapter 5 describes the hardware integration process and discusses the challenges encountered during deployment.

Chapter 6 presents the testing procedures, evaluation metrics, experimental results, and performance analysis of the system.

Chapter 7 discusses the overall findings, limitations, and effectiveness of the project.

Chapter 8 concludes the report and proposes recommendations and future improvements.

Chapter 9 provides a project management review, including reflections on planning, scheduling, challenges, and project development throughout the academic year.

Chapter 2 – Literature Review

2.1 Introduction

Research into AI-based assistive technologies for visually impaired individuals has grown considerably in recent years, with developments in computer vision, deep learning, and embedded systems making it increasingly practical to build affordable real-time assistive devices. This chapter reviews four studies that are relevant to this project, covering combined object detection and OCR systems, YOLO model comparisons, and user-based evaluations of smart vision devices. The purpose of the review is to identify what has already been done in this area, what limitations exist in current approaches, and how the present project relates to and builds on this work.

2.2 Real-Time Object and Text Recognition Using YOLO and OCR

Veeramani et al. (2025) developed a smart glasses prototype for visually impaired users that operated in two modes: an Object Detection Mode using YOLO for continuous identification of surrounding objects, and an OCR Mode that captured a still image and converted any detected text to speech using Google Text-to-Speech. The system ran on a Raspberry Pi with a webcam and was tested on objects such as people, bottles, and printed signs. The study showed that integrating YOLO, OCR and TTS into a single Raspberry Pi pipeline is achievable at prototype level and produces functional audio feedback for the user.

The architecture used by Veeramani et al. (2025) is the closest in structure to the system developed in this project, which also combines object detection, OCR and voice output on embedded hardware. However, there are notable differences. Veeramani et al. (2025) used TensorFlow with MobileNet SSD for detection and Tesseract for OCR, both of which are older approaches with known limitations in accuracy and robustness under varied conditions. This project uses YOLOv8n, which offers better detection performance at low computational cost, and EasyOCR, which handles varied fonts and lower-quality images more reliably than Tesseract. The two-mode pipeline structure described by Veeramani et al. (2025) also influenced the decision in this project to keep object detection and OCR as separate but integrated processing steps.

2.3 Comparing YOLO Model Variants for Embedded Deployment

Sahu et al. (2025) investigated the use of YOLOv11 for real-time object detection and tracking aimed at visually impaired users. Their system streamed video from a glasses-mounted camera to a mobile device running a custom-trained YOLOv11 model, with ByteTrack used for multi-object tracking. When a tracked object crossed a predefined boundary line, its name was spoken aloud through earphones via Google Text-to-Speech. The model was fine-tuned on a custom dataset that included faces, Indian currency, road potholes and facial expressions, giving it capabilities beyond what a standard pretrained COCO model would offer. Each object was detected and tracked in under 20 milliseconds (Sahu et al., 2025).

One of the more useful aspects of this paper for the present project is the comparison Sahu et al. (2025) provide across YOLO model sizes. Their results show that smaller nano-sized models are significantly faster but less accurate, while larger variants achieve higher mean average precision at the cost of considerably more processing power. This directly relates to the model selection decision in this project. Since the system is deployed on a Raspberry Pi 5 with limited computational resources, YOLOv8n was chosen as the most practical option, accepting some reduction in accuracy in exchange for acceptable inference speed on embedded hardware.

2.4 User Acceptance of AI-Powered Smart Vision Glasses

Udayakumar et al. (2025) conducted a prospective observational study in which a commercial AI-powered smart vision glasses device was given to 90 visually impaired participants across five rehabilitation centres in India to use for one month. The device provided four functions: object identification, OCR-based text reading, LiDAR-assisted walking guidance, and face recognition. Feedback was collected through a structured survey after the usage period.

The reading function was rated useful by 72.9% of participants, making it the most valued feature, followed by object identification at 44.7%, face recognition at 36.5%, and walking assistance at 22.4% (Udayakumar et al., 2025). Walking assistance also received the most requests for improvement, with 35.3% of participants identifying it as the weakest function, largely due to difficulty in low-light conditions and crowded environments. Two findings from this study are particularly relevant to the present project. First, the strong preference for OCR-based reading supports the decision to implement EasyOCR as a core component. Second, the difficulties reported with obstacle detection even in a commercial device with LiDAR sensors provides useful context for the hardware limitations encountered with the HC-SR04 ultrasonic sensor in this prototype, and shows that this is a broader challenge in the field rather than a project-specific shortcoming.

2.5 Evaluating Smart Vision Glasses with Totally Blind Users

Varshney et al. (2025) carried out a six-month prospective case series with 26 visually impaired or totally blind participants, evaluating a smart vision glasses device using semi-structured interviews, surveys, and validated questionnaires including the Difficulties in Recognising Faces and Emotions (DRFE) scale and the Expectations for Smart Glasses as Assistive Device (ESGAD) questionnaire. Testing took place at a tertiary eye care hospital in both controlled and everyday environments. Overall, 88.5% of participants rated the device as satisfactory or outstanding. Face recognition was rated very satisfactory by 61.5% of participants and the reading function was rated satisfactory by 65.4%, while only 38.5% were satisfied with walking assistance (Varshney et al., 2025). Statistical analysis confirmed significant improvements in satisfaction for reading and face recognition after device use ($p < 0.05$), but no significant improvement was found for walking assistance or currency identification. The study also found that 94% of participants had previously struggled with reading independently, and 32% had not been aware that smart glasses existed as an assistive option before the study. These findings reinforce the design priorities of this project, particularly the focus on making object detection and text reading the central functional components, and they also highlight the importance of

evaluating the system honestly against what it actually achieves rather than what was originally planned.

2.6 Summary

The four studies reviewed in this chapter demonstrate that AI-based assistive vision systems built around object detection, OCR and voice feedback are technically achievable, genuinely useful to visually impaired users, and viable at relatively low cost. A consistent theme across the literature is that navigation and obstacle detection remain the most difficult aspects to get right in practice, while reading and object identification tend to perform better and be more valued by users. Most existing systems also focus on one capability at a time rather than integrating detection, OCR and sensing into a single embedded pipeline.

This project responds to that gap by combining YOLOv8n, EasyOCR, gTTS and an HC-SR04 ultrasonic sensor within a unified system running on Raspberry Pi 5. The decisions around technology selection, system architecture and evaluation approach were all informed by the work of Veeramani et al. (2025), Sahu et al. (2025), Udayakumar et al. (2025) and Varshney et al. (2025).

Chapter 3 – System Requirements and Design

3.1 Introduction

This chapter presents the system requirements, architecture, and overall design of the proposed smart vision assistance prototype. The purpose of the design phase was to define how the software and hardware components would interact to provide object detection, text recognition, and audio feedback functionalities.

The system was designed as a low-cost embedded AI prototype capable of assisting visually impaired users through environmental awareness and spoken feedback. Both software and hardware requirements were considered during the design process to ensure compatibility with Raspberry Pi hardware and real-time processing constraints.

The chapter also discusses the functional and non-functional requirements of the system, the selected hardware and software components, and the overall system workflow.

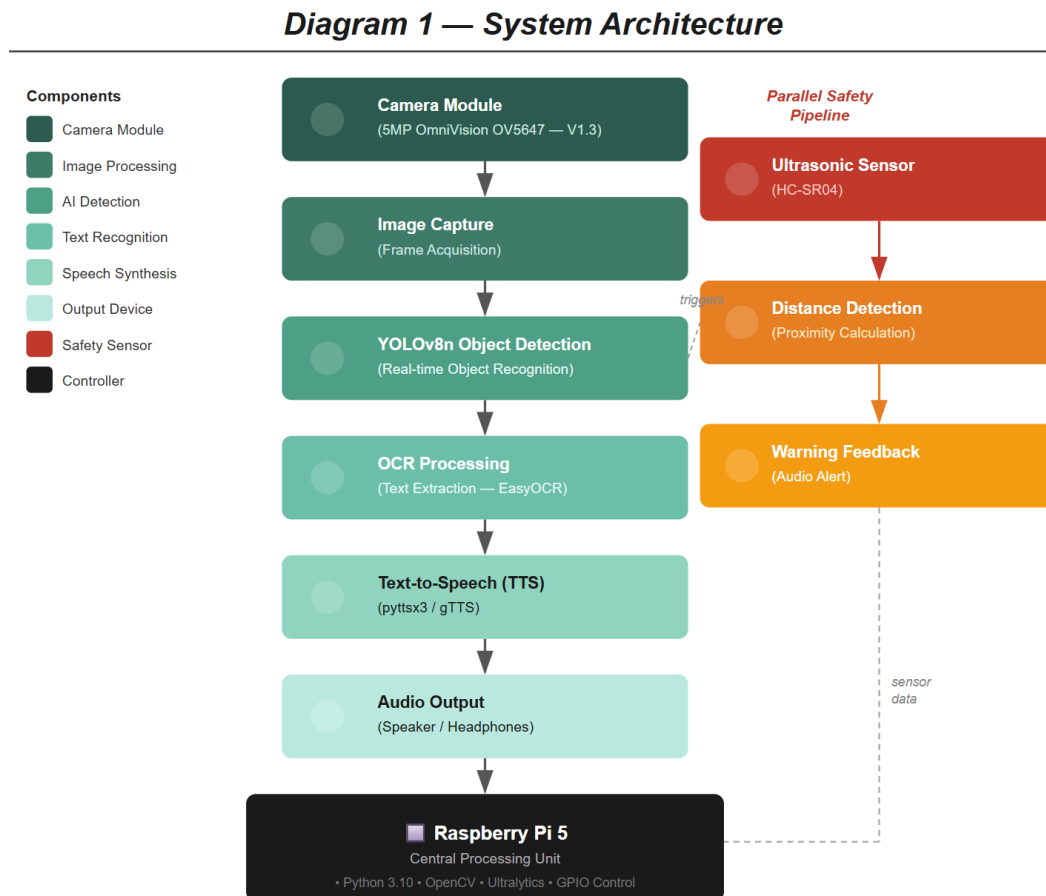


Diagram 1 — System Architecture Diagram

3.2 Functional Requirements

Functional requirements describe the core features and operations the system must perform.

The proposed system was designed to provide the following functionalities:

- Detect surrounding objects using a computer vision model.
- Recognise selected object classes relevant to indoor and outdoor environments.
- Perform Optical Character Recognition (OCR) on detected image regions.
- Convert detected objects and recognised text into spoken audio feedback.
- Capture images using a Raspberry Pi camera module.
- Detect nearby obstacles using an ultrasonic sensor.
- Process image inputs and generate outputs in near real-time.
- Operate on Raspberry Pi embedded hardware.

The system was also required to support multiple object categories, including people, vehicles, furniture, and traffic-related objects.

3.3 Non-Functional Requirements

Non-functional requirements describe the performance and usability characteristics of the system.

The system was designed with the following non-functional requirements:

- The system should provide low-cost hardware implementation.
- The software should maintain acceptable processing speed on Raspberry Pi hardware.
- The interface should remain simple and accessible for visually impaired users.
- The system should provide clear and understandable audio feedback.
- The prototype should remain lightweight and modular for future expansion.
- The object detection model should achieve reasonable detection accuracy under normal conditions.
- The software should support integration with embedded hardware components.

Several practical limitations were identified during development, including hardware processing limitations, camera integration issues, and environmental sensitivity related to lighting and image quality.

3.4 Hardware Requirements

The hardware components selected for the project were chosen based on affordability, compatibility, and suitability for embedded AI applications.

The final hardware configuration included:

Hardware Component	Purpose
Raspberry Pi 5	Main embedded processing unit
Raspberry Pi Camera Module V1.3	Capturing images for object detection and OCR
HC-SR04 Ultrasonic Sensor	Obstacle and distance detection
Breadboard	Prototype hardware assembly
Male-to-Male and Male-to-Female Wires	Circuit connections
MicroSD Card	Operating system and storage
Resistors	Voltage protection for ultrasonic sensor
Wired Headphones/Speakers	Audio feedback output

The Raspberry Pi 5 was selected due to its support for Python libraries, GPIO functionality, and compatibility with AI-based computer vision frameworks.

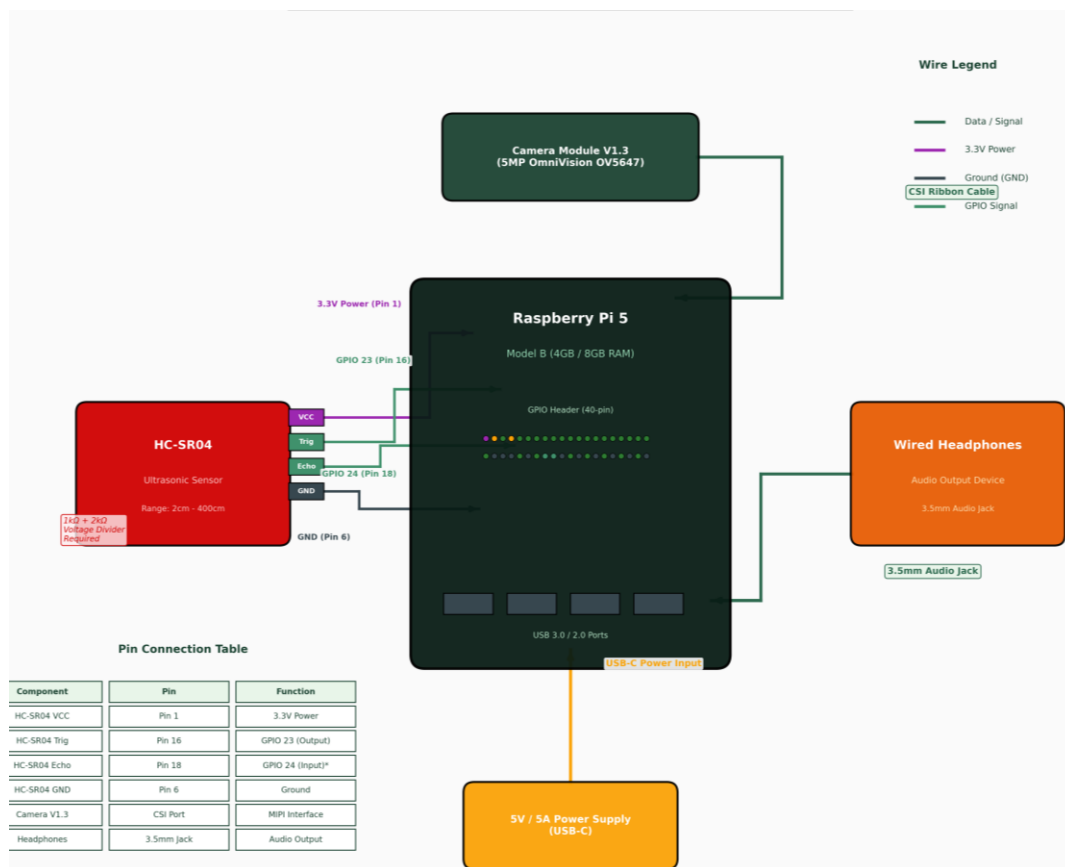


Diagram 2 — Hardware Connection Diagram

3.5 Software Requirements

The software stack used in the project focused primarily on computer vision, AI processing, and embedded system integration.

The main software technologies included:

Software/Library	Purpose
Python	Main programming language
YOLOv8	Real-time object detection
OpenCV	Image processing
EasyOCR	Optical Character Recognition
NumPy	Numerical processing
pyttsx3 / gTTS	Text-to-Speech functionality
Raspberry Pi OS	Embedded operating system

Python was selected because of its extensive support for machine learning and computer vision libraries. YOLOv8 was used for object detection due to its balance between speed and performance, while EasyOCR was integrated for text recognition functionality.

3.6 System Design

The proposed system follows a modular processing architecture consisting of image capture, object detection, OCR analysis, and audio feedback generation.

The workflow begins with image acquisition from the Raspberry Pi camera module. Captured images are processed using the YOLOv8 object detection model to identify relevant objects within the environment. Detected object regions are then analysed using OCR to extract readable text where applicable.

The extracted information is converted into speech using a text-to-speech engine, allowing the user to receive audio-based environmental feedback.

The ultrasonic sensor operates separately to detect nearby obstacles and provide additional awareness support.

The modular structure of the system allows future expansion and replacement of individual components without redesigning the entire architecture.

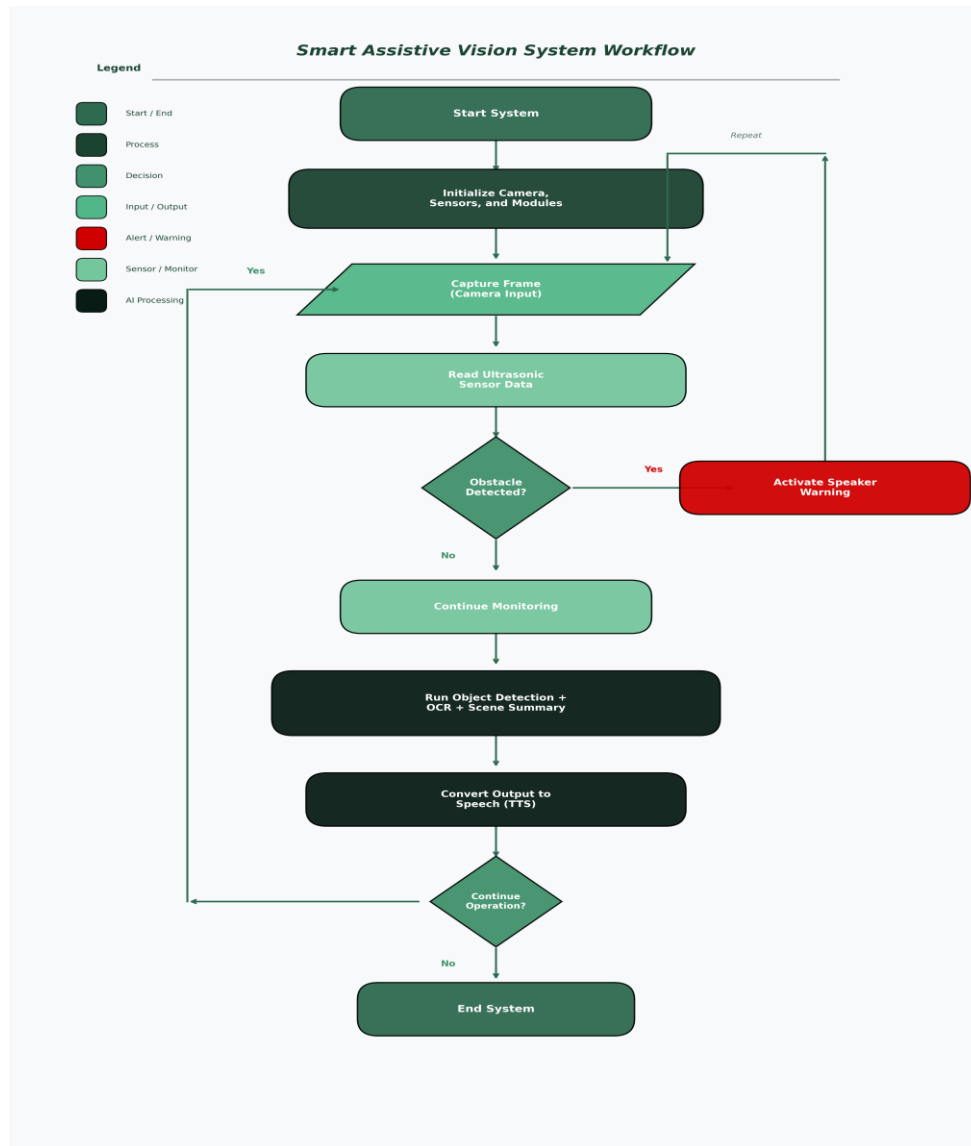


Diagram 3 — System Workflow Flowchart

3.7 Design Challenges and Constraints

Several technical and practical challenges affected the system design and implementation process.

One of the main challenges involved deploying computer vision functionality on Raspberry Pi hardware while maintaining acceptable processing performance. Lightweight AI models were therefore prioritised to reduce computational load.

Additional challenges included:

- Raspberry Pi camera integration issues
- hardware debugging difficulties
- sensor voltage protection requirements
- limited real-time testing opportunities
- environmental sensitivity related to lighting and image quality

Due to time limitations and hardware integration constraints, the final prototype was demonstrated as a table-mounted embedded system rather than a fully wearable smart glasses device.

Despite these challenges, the final prototype successfully demonstrated the integration of object detection, OCR, audio feedback, and embedded hardware components within a unified assistive system.

3.8 Summary

This chapter presented the requirements, architecture, and design of the proposed smart vision assistance prototype. Functional and non-functional requirements were identified to guide the development process, while hardware and software components were selected based on embedded AI compatibility and affordability.

The modular system architecture combined object detection, OCR, and text-to-speech technologies with Raspberry Pi hardware integration to create an assistive prototype capable of providing environmental awareness through audio feedback.

The next chapter discusses the implementation methodology and explains how the individual software and hardware components were developed and integrated into the final system.

Chapter 4 – Methodology and Implementation

4.1 Introduction

This chapter explains the methodology and implementation process used to develop the smart vision assistance prototype. The project combined computer vision, Optical Character Recognition (OCR), embedded systems, and audio feedback technologies to create an assistive system for visually impaired users.

The development process involved researching suitable AI models and software libraries, implementing object detection and OCR functionality, integrating text-to-speech feedback, and deploying the system on Raspberry Pi hardware. The chapter also discusses the technologies used, implementation decisions, and challenges encountered during development.

The overall goal of the implementation phase was to create a functional prototype capable of detecting objects, recognising text, and delivering audio-based environmental feedback.

4.2 Development Methodology

The project followed a practical iterative development methodology consisting of research, design, implementation, testing, and evaluation stages.

The first stage involved reviewing existing assistive technologies and selecting appropriate software tools and hardware components. Different object detection and OCR approaches were considered before selecting YOLOv8 and EasyOCR due to their compatibility with Python and suitability for real-time applications.

After the research stage, the system architecture and workflow were designed to define how the hardware and software components would interact. The implementation process then focused on developing each module individually before integrating them into a unified processing pipeline.

Testing and debugging were performed continuously throughout development to identify software errors, optimise system behaviour, and evaluate detection performance. Hardware integration and Raspberry Pi deployment were conducted during the later stages of the project.

The iterative methodology allowed features to be improved gradually while adapting to technical limitations and hardware integration challenges encountered during development.

4.3 Object Detection Implementation

Object detection was implemented using the YOLOv8 deep learning model. YOLOv8 was selected because it provides a balance between processing speed and detection performance, making it suitable for embedded AI applications.

The YOLOv8n (nano) model was used in this project due to its lightweight architecture and lower computational requirements compared to larger YOLO variants. This made the model more appropriate for deployment on Raspberry Pi hardware.

The model was implemented in Python using the Ultralytics YOLO framework. Captured images were passed into the model for inference, and detected objects were returned with associated class labels and confidence scores.

The system focused on detecting object categories relevant to visually impaired users, including:

- people
- vehicles
- bicycles
- traffic lights
- furniture-related objects

Detected objects were filtered based on predefined target classes before being processed further.

The following code snippet demonstrates the loading of the YOLOv8 model:

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
```

Images captured by the camera module or loaded from test datasets were processed using the model to perform object detection and generate predictions.

Confidence thresholds were used to filter weak detections and reduce false positives during evaluation.

4.4 OCR Implementation

Optical Character Recognition (OCR) functionality was implemented using EasyOCR. OCR was integrated into the system to recognise printed text from captured images and detected object regions.

EasyOCR was selected because it provides deep learning-based text recognition and integrates easily with Python applications. The OCR module was capable of extracting readable text from signs, labels, and printed materials present within the test images.

After object detection was performed, selected image regions were cropped and passed into the OCR module for text extraction.

The following code snippet demonstrates the OCR initialisation process:

```
import easyocr

reader = easyocr.Reader(['en'])
```

OCR outputs were filtered based on confidence values to reduce incorrect text recognition results.

Several factors affected OCR performance during testing, including:

- image quality
- lighting conditions
- text orientation
- motion blur
- text size

Despite these limitations, the OCR module successfully recognised text from multiple test images and demonstrated functional integration within the assistive system.

4.5 Text-to-Speech Implementation

Text-to-Speech (TTS) functionality was implemented to convert detected objects and recognised text into spoken audio feedback.

The TTS module allowed the system to communicate environmental information to the user without requiring visual interaction. Audio feedback was generated after object detection and OCR processing were completed.

The implementation used Python text-to-speech libraries to generate speech outputs dynamically during execution.

Typical spoken outputs included:

- detected object names
- warning messages
- recognised text
- obstacle notifications

The integration of TTS functionality improved the accessibility of the system by enabling hands-free audio interaction.

4.6 Image Processing and OpenCV Integration

OpenCV was used throughout the project for image processing and frame manipulation tasks. The library was responsible for:

- image loading
- image capture
- frame resizing
- cropping detected regions
- displaying processed outputs

OpenCV also supported communication between the camera module and the object detection pipeline.

The following code snippet demonstrates image loading using OpenCV:

```
import cv2

frame = cv2.imread(image)
```

The use of OpenCV simplified image preprocessing and integration between the computer vision modules.

4.7 Raspberry Pi and Hardware Integration

The system was designed for deployment on Raspberry Pi 5 hardware. Raspberry Pi was selected because it provides a compact and affordable embedded computing platform with support for Python AI libraries and GPIO connectivity.

The hardware setup included:

- Raspberry Pi 5
- Raspberry Pi Camera Module V1.3
- HC-SR04 ultrasonic sensor
- breadboard connections
- wired audio output

The Raspberry Pi camera module was intended to capture live image inputs for real-time object detection and OCR processing. The ultrasonic sensor was integrated to support obstacle awareness and distance sensing functionality.

Although the software components functioned successfully during testing, several hardware integration challenges were encountered during deployment. These included:

- Raspberry Pi camera detection issues
- configuration and compatibility problems

- sensor voltage protection requirements
- limited real-time testing opportunities

Due to these limitations, the final system was demonstrated primarily as a functional prototype rather than a fully wearable smart glasses device.

4.8 System Workflow

The complete system workflow begins with image acquisition from the Raspberry Pi camera module or test images. Captured images are processed using the YOLOv8 object detection model to identify relevant objects within the environment.

Detected image regions are then analysed using OCR to extract readable text where available. The extracted information is converted into speech using the text-to-speech module and delivered to the user through audio output.

Simultaneously, the ultrasonic sensor monitors nearby obstacles and supports environmental awareness functionality.

The modular design of the system allows future improvements and replacement of individual components without redesigning the entire system architecture.

4.9 Implementation Challenges

Several technical challenges were encountered during the implementation process.

One of the main challenges involved balancing AI processing requirements with the hardware limitations of the Raspberry Pi platform. Lightweight models and optimised processing approaches were therefore prioritised during development.

Additional challenges included:

- camera module integration issues
- inconsistent OCR performance under poor lighting conditions
- hardware debugging difficulties
- dependency installation and library compatibility
- processing delays during real-time testing

Despite these limitations, the project successfully demonstrated the integration of object detection, OCR, and audio feedback functionalities within a unified assistive system prototype.

4.10 Summary

This chapter discussed the methodology and implementation process used to develop the proposed smart vision assistance system. YOLOv8 was implemented for object detection, EasyOCR was integrated for text recognition, and text-to-speech functionality was used to generate spoken feedback.

The system was deployed using Raspberry Pi hardware and supporting peripherals, while OpenCV was used for image processing and module integration. Although several hardware and deployment challenges were encountered, the implementation successfully demonstrated the feasibility of combining AI and embedded systems for assistive technology applications.

The next chapter presents the hardware integration process in more detail and discusses the practical deployment challenges encountered during development.

Chapter 5 – Hardware Integration

5.1 Introduction

This chapter discusses the hardware integration process involved in developing the smart vision assistance prototype. The hardware implementation focused on combining embedded computing, image capture, obstacle sensing, and audio output components into a functional assistive system.

The primary goal of the hardware stage was to support the software components developed in previous chapters by enabling image acquisition, sensor input, and audio feedback through Raspberry Pi hardware. The chapter also explains the setup process, hardware configuration, integration challenges, and limitations encountered during deployment.

Although the original project concept proposed a fully wearable smart glasses device, the final implementation was demonstrated as a table-mounted functional prototype due to hardware integration constraints and time limitations.

5.2 Raspberry Pi 5 Setup

The Raspberry Pi 5 was selected as the main embedded processing unit for the system. The device was responsible for executing the object detection, OCR, and text-to-speech modules while also managing communication with connected hardware components.

The Raspberry Pi platform was chosen because it:

- supports Python-based AI frameworks
- provides GPIO connectivity
- supports camera modules and sensors
- is relatively affordable
- is suitable for embedded AI prototyping

The Raspberry Pi OS operating system was installed on a MicroSD card and configured to support the required software libraries and dependencies.

The setup process involved:

- installing Python packages
- configuring OpenCV
- installing the Ultralytics YOLO framework
- configuring OCR libraries
- testing GPIO functionality

The Raspberry Pi was connected to a laptop during development and testing to simplify debugging and software execution.

5.3 Camera Module Integration

The Raspberry Pi Camera Module V1.3 was used to provide image input for object detection and OCR processing. The camera module was connected to the Raspberry Pi using the CSI ribbon cable interface.

The intended purpose of the camera module was to:

- capture live environmental images
- support real-time object detection
- provide OCR image input
- enable portable assistive functionality

During implementation, several challenges were encountered related to camera integration and image capture functionality. Although the software system itself executed correctly, the camera module experienced configuration and detection issues during Raspberry Pi deployment.

Possible causes included:

- camera configuration settings
- ribbon cable orientation
- compatibility issues
- software pipeline configuration problems

As a result, testing was primarily conducted using stored test images while camera debugging continued during the later stages of the project.

Despite these challenges, the software components remained fully functional and compatible with image-based inputs.

5.4 Ultrasonic Sensor Integration

An HC-SR04 ultrasonic sensor was integrated into the system to support obstacle awareness and distance detection functionality.

The ultrasonic sensor operates by transmitting ultrasonic sound waves and measuring the time required for the reflected signal to return. This allows the system to estimate the distance between the sensor and nearby obstacles.

The sensor was connected to the Raspberry Pi GPIO pins using a breadboard and jumper wires. Resistors were required to provide voltage protection and ensure safe GPIO operation.

The intended functionality of the sensor included:

- obstacle detection
- distance awareness
- warning generation
- assistive environmental feedback

The sensor setup was visually assembled and partially connected during development. However, full real-time sensor testing was limited due to missing resistor components and time constraints during hardware integration.

Despite incomplete live testing, the hardware architecture and software integration logic for the ultrasonic sensor were successfully prepared.

5.5 Audio Output Integration

Audio feedback was implemented to communicate object detection and OCR outputs to the user. Spoken feedback was generated using text-to-speech libraries integrated within the software system.

Audio output was delivered using connected speakers or wired headphones during testing and demonstration stages. The use of audio feedback was essential for accessibility because it allowed users to receive information without visual interaction.

Typical spoken outputs included:

- detected object names
- recognised text
- warning notifications
- environmental feedback

The integration of audio output functionality significantly improved the usability and accessibility of the system prototype.

5.6 Hardware Assembly and Prototype Design

The original project proposal aimed to create a wearable smart glasses device integrating AI-based assistive technologies. However, due to hardware debugging challenges and limited development time, the final implementation focused on creating a functional prototype demonstrating the complete system architecture.

The prototype was assembled using:

- Raspberry Pi hardware
- breadboard connections
- camera module
- ultrasonic sensor
- wired peripheral connections

The final prototype was demonstrated as a table-mounted configuration rather than a physically wearable device. This approach allowed the system functionality and hardware integration process to be tested without requiring complete wearable assembly.

The ultrasonic sensor setup was partially connected during hardware assembly. Full deployment of the HC-SR04 sensor required resistor-based voltage protection for safe Raspberry Pi GPIO operation, and complete real-time testing could therefore not be finalised during the current development stage. In addition, wired audio output devices such as headphones were prepared for integration but were not fully connected during the current hardware implementation phase.

The modular hardware structure also allows future expansion into a wearable form factor if additional development time and hardware refinement are available.

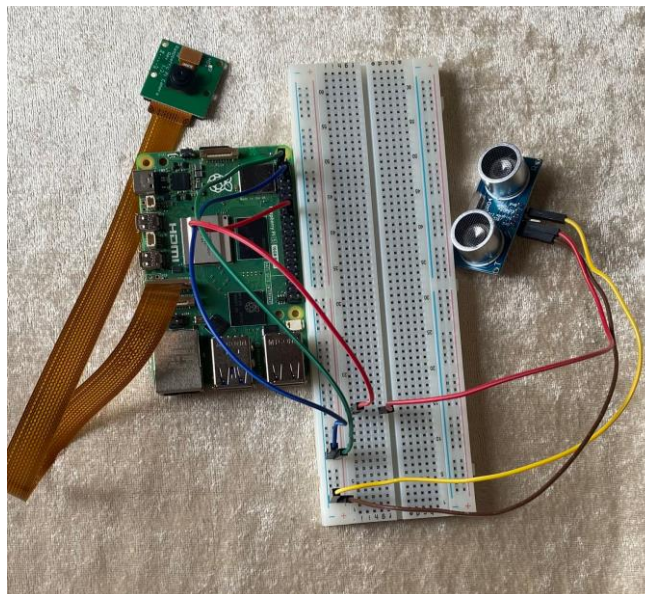


Figure 5.4 – Prototype Hardware Assembly

5.7 Hardware Integration Challenges

Several hardware-related challenges were encountered during the implementation process.

One of the primary challenges involved integrating the Raspberry Pi camera module with the object detection pipeline. Although the AI software components functioned correctly, live image capture on the Raspberry Pi platform required additional debugging and configuration.

Additional hardware challenges included:

- GPIO voltage protection requirements
- missing resistor components during sensor testing
- hardware compatibility troubleshooting
- embedded processing limitations
- cable management and prototype portability

The limited processing power of embedded hardware compared to desktop systems also affected real-time performance expectations during deployment testing.

These challenges highlighted the practical difficulties associated with combining AI-based computer vision systems with embedded hardware platforms.

5.8 Hardware Limitations

Several limitations affected the final hardware implementation.

The system was not fully deployed as a wearable smart glasses device due to:

- time limitations
- hardware debugging challenges
- camera integration issues
- incomplete real-time sensor testing

In addition, environmental conditions such as lighting quality and camera positioning affected the reliability of image-based processing.

The Raspberry Pi hardware also introduced processing constraints compared to higher-performance desktop environments, which may impact real-time detection performance in practical deployment scenarios.

Despite these limitations, the final prototype successfully demonstrated the integration of embedded hardware, AI-based object detection, OCR functionality, and speech-based feedback within a unified assistive system.

5.9 Summary

This chapter discussed the hardware integration process used to support the smart vision assistance prototype. Raspberry Pi hardware, a camera module, an ultrasonic sensor, and audio output devices were integrated to provide embedded AI functionality and environmental awareness features.

Although several deployment and hardware debugging challenges were encountered, the final prototype successfully demonstrated the feasibility of combining AI software systems with embedded hardware components for assistive technology applications.

The next chapter presents the testing procedures, evaluation methods, and experimental results used to assess the performance of the proposed system.

Chapter 6 – Testing, Results, Discussion and Evaluation

6.1 Introduction

This chapter presents the testing procedures, experimental results, and evaluation of the proposed smart vision assistance system. The purpose of testing was to evaluate the functionality and performance of the integrated object detection, OCR, and text-to-speech modules under different conditions.

Testing focused primarily on:

- object detection performance
- OCR functionality
- processing efficiency
- system integration
- hardware deployment behaviour

The evaluation process combined both quantitative metrics and qualitative observations in order to assess the effectiveness of the prototype.

6.2 Testing Methodology

Testing was conducted using a collection of indoor and outdoor images containing multiple object categories relevant to the assistive system.

The test dataset included:

- people
- vehicles
- furniture
- traffic-related objects
- printed text samples

The YOLOv8 model processed each image to identify target objects, while EasyOCR was used to detect and extract readable text. The generated outputs were then converted into spoken feedback using the text-to-speech module.

Performance evaluation focused on:

- precision
- recall
- F1-score
- confidence scores

- OCR functionality
- processing behaviour

Testing was performed primarily using laptop-based execution and Raspberry Pi simulation due to ongoing hardware integration challenges during live deployment testing.

6.2.1 Target Detection Classes

The object detection system focused on identifying object categories considered important for assistive navigation and environmental awareness.

The selected target classes were obtained from the COCO dataset categories supported by YOLOv8 and included:

Target Classes
Person
Car
Bus
Bicycle
Traffic Light
Chair
Couch
Dining Table
Bed
TV

These object categories were selected because they represent common environmental elements that may assist visually impaired users during navigation and daily activities.

6.3 Object Detection Results

6.3.1 Evaluation Metrics

The performance of the object detection system was evaluated using precision, recall, and F1-score metrics. These metrics were selected because they provide a balanced evaluation of detection accuracy and reliability.

Precision measures the proportion of correctly detected objects compared to the total number of detections generated by the model.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \times 100$$

Recall measures the proportion of correctly detected objects compared to the total number of relevant objects present within the test dataset.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \times 100$$

The F1-score combines precision and recall into a single performance metric and provides a balanced measure of detection effectiveness.

$$\text{F1-score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Where:

- TP represents True Positives
- FP represents False Positives
- FN represents False Negatives

The YOLOv8n object detection model successfully identified multiple target objects across the testing dataset. The system generated bounding boxes, class labels, and confidence scores for detected objects while filtering outputs according to the predefined target classes.

The model demonstrated consistent detection performance across both indoor and outdoor test images, particularly for commonly occurring environmental objects such as people, vehicles, and furniture-related categories.

The following figure demonstrates an example object detection result generated during testing.

The average confidence score of 56.6% reflects moderate confidence across mixed indoor and outdoor environments, which is expected when using the lightweight YOLOv8n model on varied real-world images.

6.4 OCR Testing Results

OCR testing was conducted using multiple text-containing images in order to evaluate the ability of the system to recognise readable text from captured scenes.

The OCR module successfully recognised several text samples, including:

- warning signs
- printed labels
- simple text phrases

The system extracted readable text and passed the results into the text-to-speech module for spoken feedback generation.

Examples of recognised text included:

- “NO ENTRY”
- “Noisy image to test Tesseract OCR”

The OCR system detected a total of eight text elements during testing.

OCR performance was strongly influenced by:

- image quality
- lighting conditions
- text orientation
- background noise
- image sharpness

Clear and well-lit images produced significantly better OCR results compared to blurred or low quality inputs.



Figure 6.2 – OCR Recognition Result

6.5 Text-to-Speech Evaluation

The text-to-speech module was evaluated based on its ability to convert detected outputs into understandable spoken feedback.

The TTS functionality successfully generated audio output for:

- object detection labels
- recognised OCR text
- environmental notifications

Testing confirmed that spoken feedback could be generated dynamically after image processing and detection stages were completed.

The integration of TTS functionality improved the accessibility of the prototype by allowing environmental information to be delivered without requiring visual interaction from the user.

During testing, audio output was delivered through connected speakers or headphones.

6.6 Hardware Testing and Deployment Behaviour

Hardware testing focused on evaluating the interaction between the Raspberry Pi hardware components and the AI software modules.

The Raspberry Pi 5 successfully supported:

- Python execution
- AI library installation

- object detection processing
- OCR integration
- GPIO preparation

The ultrasonic sensor was connected and partially integrated into the hardware setup. However, full real-time sensor testing was limited due to missing resistor components required for safe voltage protection.

The Raspberry Pi camera module was connected physically, but live image capture functionality experienced configuration and compatibility issues during deployment testing.

Although these hardware challenges limited full real-time operation, the overall software pipeline remained functional using stored image inputs and simulated testing environments.

6.7 Discussion

The testing results demonstrate that the proposed system successfully integrated object detection, OCR, and text-to-speech functionalities into a unified assistive prototype.

The object detection module achieved strong recall performance and demonstrated the ability to recognise multiple object categories relevant to assistive navigation scenarios. The OCR module also showed functional text recognition capabilities under suitable image conditions.

One of the main strengths of the system was the successful integration of multiple AI technologies within a lightweight embedded system architecture.

However, several limitations affected the overall system performance.

The lightweight YOLOv8n model prioritised processing efficiency over maximum detection accuracy, which contributed to occasional false positives and moderate confidence scores.

Additional limitations included:

- hardware integration difficulties
- camera deployment issues
- environmental sensitivity
- incomplete real-time testing
- Raspberry Pi processing constraints

The OCR module was also sensitive to image quality and performed less effectively under noisy or poorly lit conditions.

Despite these limitations, the project successfully demonstrated the feasibility of combining computer vision and embedded systems to support assistive technology applications.

6.8 System Evaluation

Overall, the proposed prototype achieved its primary objectives by:

- detecting relevant environmental objects
- recognising text from images
- generating spoken audio feedback
- integrating AI software with embedded hardware

The software components performed successfully during testing and produced measurable outputs suitable for assistive applications.

Although the hardware deployment was not fully completed as a wearable smart glasses system, the implemented prototype demonstrated the core functionality and system architecture proposed during the project planning stage.

The project therefore achieved a functional proof-of-concept implementation while also identifying important areas for future improvement and optimisation.

6.9 Limitations

Several limitations affected the final implementation and evaluation process.

These limitations included:

- incomplete real-time camera integration
- limited embedded hardware testing
- dependency on lighting conditions
- limited testing dataset size
- lightweight AI model constraints
- time limitations during deployment

The project also relied on manually evaluated test images rather than large-scale benchmark datasets, meaning that the evaluation focused more on prototype functionality than research-grade AI benchmarking.

Despite these limitations, the project still demonstrated successful integration of the major system components.

6.10 Summary

This chapter presented the testing procedures, evaluation results, and performance analysis of the proposed smart vision assistance system.

The YOLOv8 object detection model achieved an F1-score of 81.3%, while the OCR and text-to-speech modules demonstrated successful functionality during testing. The system also showed successful integration between AI software modules and Raspberry Pi hardware components.

Although several hardware deployment challenges and limitations were encountered, the overall prototype successfully demonstrated the feasibility of AI-assisted smart vision technology for visually impaired support applications.

The next chapter presents the final conclusions, recommendations, and future development opportunities for the proposed system.

Chapter 7 – Conclusions and Future Work

7.1 Conclusion

This project aimed to design and develop an AI-based smart assistive vision system capable of supporting visually impaired users through object detection, optical character recognition (OCR), and audio feedback functionalities. The proposed system combined computer vision technologies with embedded hardware components to create a lightweight assistive prototype using the Raspberry Pi platform.

The final implementation successfully integrated the YOLOv8n object detection model, EasyOCR text recognition, and text-to-speech functionality into a single system capable of analysing environmental scenes and generating spoken feedback. Testing results demonstrated that the system was able to detect relevant objects with satisfactory performance while maintaining processing efficiency suitable for embedded deployment.

The evaluation process showed that the object detection component achieved a precision score of 73.6%, a recall score of 90.7%, and an F1-score of 81.3% across the selected test dataset. In addition, the OCR module successfully extracted readable text from multiple test images, while the text-to-speech component generated understandable audio feedback based on detected objects and recognised text.

Several technical challenges were encountered during the development process, particularly during hardware integration and Raspberry Pi camera deployment. Although the software components operated successfully, full real-time camera integration required additional debugging and optimisation. Despite these limitations, the project successfully demonstrated the feasibility of combining artificial intelligence, computer vision, and accessibility technologies into a functional assistive prototype system.

Overall, the project achieved its primary objectives and demonstrated how AI-based technologies can contribute towards improving environmental awareness and accessibility support for visually impaired users.

7.2 Future Work

Although the developed prototype achieved successful software integration and functional testing, several improvements could be implemented in future development stages.

One possible improvement would involve completing full real-time wearable deployment using properly mounted smart glasses hardware rather than a table-mounted prototype configuration. This would improve portability and provide a more realistic user experience during navigation tasks.

Future work could also include improving the Raspberry Pi camera integration process to support continuous live object detection and real-time environmental analysis. Additional optimisation techniques may help improve processing speed and reduce latency during deployment on embedded hardware devices.

Another improvement would involve expanding the object detection dataset and training a custom YOLO model specifically designed for assistive navigation environments. This could increase detection accuracy and improve recognition performance for objects commonly encountered by visually impaired users.

The OCR component could also be enhanced by improving text recognition under low-light conditions, angled perspectives, and partially obscured text scenarios. In addition, multilingual OCR support could be implemented to improve usability across different languages and environments.

Further development could include the integration of advanced navigation assistance features such as GPS tracking, obstacle distance estimation, and voice-command interaction. User testing involving visually impaired participants could also provide valuable feedback regarding usability, accessibility, and system effectiveness within real-world environments.

Overall, the project provides a strong foundation for future development of intelligent assistive technologies that combine artificial intelligence and embedded systems to support accessibility and independent navigation.

Chapter 8 – Project Management Review

8.1 Introduction

This chapter reflects on how the project was planned and managed from October 2025 through to submission in May 2026. It compares the original Gantt chart submitted with the Project Outline against what actually happened, explains where the plan changed and why, and reflects honestly on personal strengths and weaknesses as a project manager. Both Gantt charts are included in Appendix A for direct comparison.

8.2 Project Planning and Time Management

The original project plan divided the work into eight stages: background research, system design, software implementation, IoT and hardware integration, system optimisation, testing and evaluation, documentation, and presentation. The plan assumed software would be complete by the end of November 2025, with hardware integration running from December 2025 onward.

In practice this did not happen. Software implementation extended into January 2026, roughly six weeks later than planned. The main reasons were dependency conflicts between YOLOv8, EasyOCR and OpenCV that took longer to resolve than expected, and the additional time needed to build and test the Raspberry Pi simulation mode. Hardware integration did not begin until late February 2026, again later than planned, because resistors required for the HC-SR04 voltage divider circuit were not obtained until that point. This meant full sensor testing and live camera deployment could not be completed before the submission deadline.

Report writing also overlapped with implementation work rather than following it sequentially as the original plan assumed, which added pressure during the final weeks.

8.3 Challenges Encountered

The most significant technical challenge was getting the Raspberry Pi camera to produce output. The Pi 5 uses libcamera by default and does not support the older picamera Python library without additional configuration. OpenCV's VideoCapture function, used in the project code, requires a specific GStreamer pipeline string to work with libcamera, which was not configured in time. The camera was physically connected and the software ran without errors, but no frames were captured. This was the single biggest gap between what was planned and what was delivered.

The missing resistors were a smaller but still impactful issue. Without the voltage divider, connecting the HC-SR04 echo pin directly to the Raspberry Pi GPIO risked damaging the board, so sensor testing had to wait. Once the resistors arrived in late February, the time available for hardware testing was limited.

Managing the workload across implementation, hardware debugging, testing and report writing simultaneously in the final two months was also difficult, and in hindsight more time should have been allocated to hardware earlier in the project.

8.4 Adaptations and Changes to the Original Plan

The most significant change from the original proposal was dropping the fully wearable form factor. The original concept described smart glasses with all components physically mounted on a glasses frame. This was not achieved. Instead the final prototype was demonstrated as a bench mounted system with the Raspberry Pi, camera and sensor connected on a flat surface. This decision was made deliberately once it became clear that resolving the camera issue and completing the software evaluation was more important than the physical form factor for demonstrating the system's functionality.

The scope of testing also shifted. Rather than testing with a live camera feed, all evaluation was done on six static test images. This was a limitation, but it still allowed meaningful evaluation of the detection pipeline, OCR module, and TTS output.

8.5 Reflection on Personal Performance

One clear strength throughout the project was the decision to build in a simulation mode for the Raspberry Pi code early on. This meant hardware delays never blocked software progress — the full pipeline could be tested on a laptop even when the Pi was not available. That was a good technical decision that saved time.

A clear weakness was leaving hardware procurement too late. The resistors were a small, cheap component that should have been ordered in October alongside everything else. Waiting until February to discover they were missing was avoidable. Similarly, the camera compatibility issue with libcamera on the Pi 5 was something that could have been identified earlier with more research into the Pi 5 specifically, rather than assuming the same setup that works on Pi 4 would carry over.

If the project were repeated, hardware setup and compatibility testing would be the first priority, not something left until after the software was finished.

8.6 Chapter Summary

The project broadly followed its planned structure but ran later than scheduled, particularly on the hardware side. The core software objectives were met, and the evaluation produced valid results. The main unresolved issue is live camera capture on the Raspberry Pi 5 it has a clear technical explanation and a clear solution for future work. The project provided real experience of managing a multi-component AI and hardware system under time pressure, and the lessons around planning and hardware procurement are ones that would directly inform how a similar project would be approached in the future.

References

Jaider AI, 2024. *EasyOCR documentation*. [online] Available at: <https://www.jaider.ai/easyocr> [Accessed 1 November 2025].

Python Software Foundation, 2024. *pyttsx3: Text-to-speech conversion library for Python*. [online] Available at: <https://pypi.org/project/pyttsx3> [Accessed 1 November 2025].

Raspberry Pi Foundation, 2024. *Raspberry Pi 5 technical specifications*. [online] Available at: <https://www.raspberrypi.com> [Accessed 1 November 2025].

Sahu, T., Padhy, N., Panigrahi, R. and Rajguru, B.K., 2025. Object detection and tracking for the visually impaired with YOLO 11. In: *Proceedings of the 2025 International Conference on Emerging Systems and Intelligent Computing (ESIC)*. IEEE, pp.648–652.

Udayakumar, D., Gopalakrishnan, S., Raghuram, A., Kartha, A., Krishnan, A.K., Ramamirtham, R., Muthangi, R. and Raju, R., 2025. Artificial intelligence-powered smart vision glasses for the visually impaired. *Indian Journal of Ophthalmology*, 73(S3), pp.S492–S497.

Ultralytics, 2024. *YOLOv8 documentation*. [online] Available at: <https://docs.ultralytics.com> [Accessed 1 November 2025].

Varshney, A.S., Chougale, M.E., Patel, C.V. and Chauhan, M.D., 2025. Evaluating usability of 'the smart vision glasses' for individuals who are visually impaired and totally blind. *Saudi Journal of Ophthalmology*. https://doi.org/10.4103/sjopt.sjopt_241_24.

Veeramani, P., Kayalvizhi, S., Sampaul, K.S., Vallinayagam, G. and Yugesh, R.S., 2025. Real time assistance for visually impaired using text and object recognition. In: *Proceedings of the 2025 International Conference on Intelligent Computing and Control Systems (ICICCS)*. IEEE, pp.989–995.

Wang, C-Y., Bochkovskiy, A. and Liao, H-Y.M., 2024. YOLOv10: You only look once again. *arXiv preprint arXiv:2405.14458*.

World Health Organization, 2023. *World report on vision*. Geneva: WHO Press.

APPENDIX A: Gantt Chart

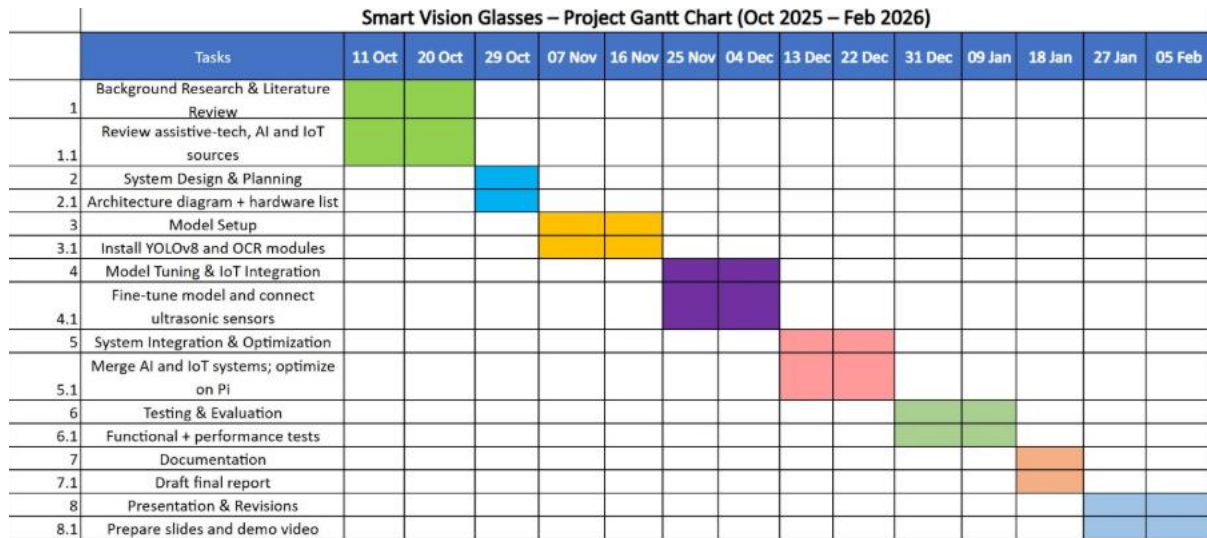


Figure A.1 – Original Gantt Chart (November 2025) (original Gantt image)

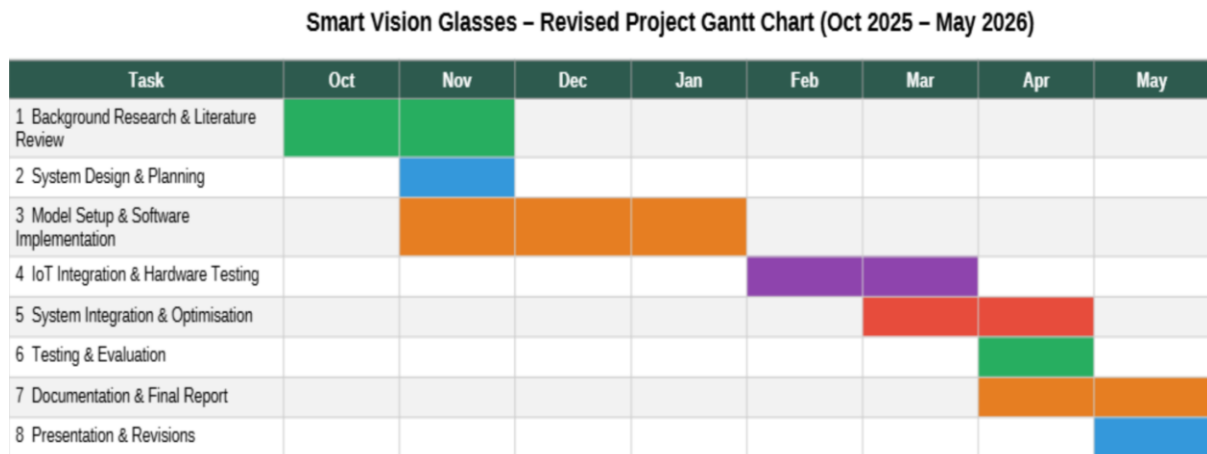


Figure A.2 – Revised Gantt Chart (May 2026) (revised Gantt table)

Appendix B: Project Resources Checklist

Student's Name: Hanin Osama Saad Hussein

SRN: 22038299

Project Title: Smart Vision Glasses for the Visually Impaired

Supervisor's Name: Dr Lamees Nasser

Date: November 2025

Question	YES / NO
1. Will a specialist laboratory PC be required for the project?	NO
2. If the answer to 1 is Yes, will you require any special hardware installed/attached? If YES, please specify below:	NO
3. Do you require access to specialist systems and/or hardware (Database Servers, Network Hardware, Robotics, etc)? If YES, please specify below: Small-scale IoT hardware components including a Raspberry Pi 5, ultrasonic sensors, a camera module, and speaker for testing the prototype.	YES
4. Will you require specific software? If YES, please specify below: Python 3.10+, YOLOv8 (Ultralytics), EasyOCR, OpenCV, pyttsx3/gTTS, VS Code.	YES
5. Are there other resources required, not covered above?	NO

Appendix C: Ethics Checklist

Student's Name: Hanin Osama Saad Hussein

SRN: 22038299

Project Title: Smart Vision Glasses for the Visually Impaired

Supervisor's Name: Dr Lamees Nasser

Date: November 2025

Ethical Issue	YES / NO
1. Will your work involve human subjects, e.g., survey, interview, experiment, etc. This includes friends and family.	NO

As the answer to question 1 is NO, formal Ethics Committee approval was not required for this project. All testing was conducted using static images with no involvement of human participants.

Appendix D: Risk Register

Student's Name: Hanin Osama Saad Hussein

SRN: 22038299

Project Title: Smart Vision Glasses for the Visually Impaired

Supervisor's Name: Dr Lamees Nasser

Date: November 2025

Task	Associated Risk	Mitigation Process	Likelihood	Severity
Background Research & Literature Review	Difficulty finding up to date or relevant research on wearable assistive AI systems	Use multiple databases (IEEE Xplore, Google Scholar); broaden search terms	Low	Low
System Design & Planning	Over-complicated architecture or unclear component links	Review design early with supervisor; simplify data flow and keep modular structure	Medium	Medium
Model Setup & Software Implementation	Installation or dependency conflicts between YOLOv8, OCR and TTS libraries	Use a virtual environment; document versions; test modules separately	Medium	Medium
IoT Integration & Hardware Testing	Sensor wiring errors or hardware failure	Test each sensor individually; keep spare components; verify voltage/pin setup	Medium	High
System Integration & Optimisation	Slow inference speed on Raspberry Pi hardware	Optimise model (quantization / Tiny YOLO); reduce	High	Medium

		image size; adjust frame rate		
Testing & Evaluation	Inconsistent results due to lighting or environment	Conduct tests in varied lighting; repeat under controlled conditions	Medium	Medium
Final Documentation & Report	Time pressure near submission causing incomplete documentation	Set internal deadlines; update report weekly; use backups	Medium	Low
Presentation & Revisions	Hardware or software failure during live demo	Prepare backup video demo and screenshots; test presentation setup beforehand	Low	Medium

Risk Summary: The main risks involve Raspberry Pi performance and sensor reliability. Risks were mitigated through modular testing, hardware backups, and frequent supervisor reviews.